SPARXSTAR
INTEGRITY MAP

SPARXSTAR

Platform Integrity Map

The Normative Superspec

Version 1.0 · Starisian Technologies

Supersedes contradictions between: Sirus v2.0 · Helios v1.0 · Sky v1.0 · Mḗhₒns v3.0 · Dheghom v1.0

This document sits above all product specifications. Where any product spec conflicts with a rule in this document, this document governs. Product specs are updated to match — they do not override this map.

CONFIDENTIAL · PATENT PENDING

How to Read This Document

Each rule in this document is normative. Rules are labelled by section and number (e.g., Rule 1.1). Decisions are frozen values that close a previously open item — they are labelled DECISION and carry the same normative weight as rules.

Items still genuinely open are labelled [OPEN]. They remain in this document so their status is visible in one place. When an [OPEN] item is resolved, this document is updated first, then the affected product spec.

The five product specifications — Sirus, Helios, Sky, Mḗh₁n̥s, Dheghom — remain the authoritative technical references for their respective components. This document resolves contradictions between them and freezes cross-component decisions that no single product spec can own alone. Section 0 (Commercial Plugin Hostility) is the foundational assumption under which every rule in this document and every product specification is written. Read it first. Every "must" in this document means: must hold in a hostile commercial plugin environment.

0. Foundational Assumption — Commercial Plugin Hostility

This section precedes all numbered rules. Every rule in this document, and every rule in every product specification this document governs, is written under this assumption. It is not a constraint specific to any one component. It applies to all six repositories, all tooling, all integrations, and all future code written against this platform.

0.1 The Hostile Environment Assumption

The SPARXSTAR platform is deployed in a full commercial WordPress ecosystem. That ecosystem includes WooCommerce extensions, SEO tools, post grid builders, AI plugins, admin tools, importers, exporters, theme frameworks, snippet managers, and an uncountable number of third-party integrations — present now and added in the future by operators who do not know this platform exists.

Any one of these plugins may, at any time and without warning:

- Call update_post_meta() or add_post_meta() against any meta key, including aiwa_ and sparxstar_ prefixed keys
- Execute WP_Query calls that return draft, quarantined, or encrypted records
- Hook into WordPress filters and actions at any priority, including priorities used by this platform
- Register REST endpoints in the /sparxstar/ or /aiwa/ namespaces
- Interact with WordPress authentication in ways that create session state
- Write directly to the database via $wpdb->query() bypassing all WordPress hook layers
- Import, export, or duplicate post records including their meta fields
- Modify post_status of any post including governed records

None of these behaviors require malicious intent. They are normal, expected behaviors of well-functioning commercial WordPress plugins doing exactly what they were designed to do.

Rule 0.1 — Third-Party Plugin Non-Trust: No SPARXSTAR security, governance, sovereignty, or archival guarantee may depend on the behavior, restraint, or compatibility of third-party WordPress plugins, themes, snippets, or tools. All such guarantees must be enforceable even when other plugins are present and operating normally. If a guarantee depends on other code cooperating, it is not a real guarantee.

Rule 0.2 — Host Runtime vs Sovereign Runtime: WordPress is treated as a host runtime for UI, routing, REST API registration, and third-party integration. Dheghom is the sovereign runtime for governed data. When these two concerns conflict — when WordPress would allow something that sovereign data governance prohibits — the sovereign runtime governs. WordPress does not override Dheghom. Dheghom enforces its contracts regardless of what WordPress or any plugin would otherwise permit.

Rule 0.3 — Architecture Enforces, Etiquette Does Not: Sovereign guarantees are enforced by architecture: hook interception, cryptographic token requirements, envelope encryption, triple binding, and database-level permission restrictions. They are not enforced by plugin etiquette, developer convention, documentation, or the assumption that other plugins will not touch governed fields. Documentation describes the system. Architecture enforces it.

0.2 What Commercial Plugin Hostility Means in Practice

- Governed data cannot rely on ordinary wp_postmeta protection alone — WordPress meta is world-writable by default to any authenticated code
- Field prefixes (aiwa_, sparxstar_) are classification markers, not access controls — any plugin can write to a prefixed key without knowing or caring what the prefix means
- WordPress hook interception (DatabaseWriteInterceptor, MetadataReadInterceptor) is a necessary layer but not the final layer — raw $wpdb->query() bypasses all PHP-level hooks entirely
- The final layer is database-level: the MySQL/MariaDB user account for the WordPress installation must have INSERT-only permissions on wp_postmeta rows for aiwa_ and sparxstar_ keys — UPDATE and DELETE must be denied at the database engine level, not the PHP level
- Any data that truly requires sovereign guarantees lives in storage controlled and validated by Dheghom — WordPress is the presentation and routing layer for that data, not its authority
- This assumption applies to all six repositories: Sirus, Helios, Sky, Méh₁n̥s, Dheghom, and Ouroboros Integrity. It applies to all tooling, all CI pipelines, all staging environments, and all production deployments

0.3 The Consequence for Every Developer

When you write code for this platform, you are not writing code for a clean controlled environment. You are writing code that must hold its guarantees while surrounded by arbitrary third-party behavior. That means:

- Never assume a governed field has not been touched by external code — always verify
- Never assume WordPress returned what you expect — always validate through the DAL
- Never assume a REST endpoint is only called by known clients — always enforce Helios agreement
- Never assume the boot order has been respected — the loader enforces it, but write code defensively as if it might not have been
- Never assume encryption is intact — always decrypt and verify before trusting draft data
- Never assume post_status reflects actual governance state — always verify the governance_token

The system is designed to enforce its own guarantees without relying on the environment being controlled. Every enforcement mechanism in this document and in the product specifications exists because the environment cannot be controlled — only the code can.

All numbered rules that follow are written under this assumption. When a rule says "must" or "must not," it means: must hold even in the presence of a hostile commercial plugin environment. If an implementation satisfies a rule only when other plugins cooperate, the rule is not satisfied.

1. Resolution of the Session Contradiction

The Helios specification states that Helios must not create WordPress sessions for frontend users and must not interact with WordPress authentication. The Sky specification previously described Helios as creating WordPress sessions. That contradiction is resolved here permanently.

Rule 1.1 — Auth Isolation: Standard WordPress authentication — the wp-login.php path and wp_set_auth_cookie() — is strictly reserved for administrative staff and the wp-admin surface. It is never used for frontend users.

Rule 1.2 — Frontend Statelessness: Helios must not create, manage, or interact with native WordPress core sessions or cookies for frontend users (Elders, contributors, musicians, community members). Helios issues and evaluates the Sirus ContextPulse. That is the extent of its session involvement.

Rule 1.3 — Sky "Session" Definition: Any reference to a "session" within the Sky or Dheghom specifications refers exclusively to the custom wp_spx_sessions database table. This is a conversation accumulation store, not a PHP session or WordPress authentication session. These two things share a name and nothing else.

Rule 1.4 — The Continuity Handshake: Sky maintains continuity by passing the Sirus-issued ContextPulse (HttpOnly, SameSite=Strict cookie) with every request. The backend identifies session continuity by the Sirus-issued device_id from the ContextPulse and resolves authenticated identity from the Helios-verified request context, then matches those against the custom wp_spx_sessions store. The ContextPulse does not carry identity claims — it carries device state, trust score, behavior flags, geo zone, and session duration only.

Note: The ContextPulse intentionally excludes identity claims. Identity resolution requires the broader Sirus context or a Helios-verified request context, not the Pulse alone. Any implementation that reads identity_id from the Pulse is incorrect.

2. Boot and Orchestration Sequence

The platform has a defined boot order that is architecturally necessary and mechanically enforced. No component may assume context established by a component that has not yet loaded.

Rule 2.1 — Sirus Is the Platform Kernel: Sirus must load as a WordPress Must-Use (mu-plugin) at the highest priority. It is responsible for establishing Environment and Device context before any other plugin, theme, or application logic executes. Sirus cannot be deactivated by any site administrator.

Rule 2.2 — Helios Is the Gatekeeper: Helios must load after Sirus and before any application module. It consumes the ContextPulse produced by Sirus to evaluate whether the current request may proceed.

Rule 2.3 — Application Modules Boot After Helios: Sky, Méh₁n̥s, and Dheghom may load after Helios in the application stack. No governed execution path — release, commit, structured data write — may proceed unless Helios has verified the Agreement State. Draft accumulation and session capture (wp_spx_sessions) are permitted before Agreement is verified; release and publish are not.

Rule 2.4 — Mechanical Enforcement: The boot order is enforced by a single loader file: mu-plugins/00-sparxstar-loader.php. This file explicitly requires components in the following order: Sirus → Helios → Sky → Méh₁n̥s → Dheghom. No component is required directly by another — all requirements flow through the loader. If any component is loaded outside 00-sparxstar-loader.php, the system must throw a BootIntegrityException and halt execution immediately with exit(1). The halt must occur before WordPress continues its boot sequence. It must not rely on wp_die(), theme rendering, or any REST fallback — those require WordPress to be partially booted, which means plugins may already have run and partial state may have leaked. Hard stop only. If the loader fails, the system does not exist.

The Boot Sequence

```
mu-plugins/00-sparxstar-loader.php

  1. require Sirus         → Environment + Device context established
     ContextEngine::current() becomes available

  2. require Helios        → Agreement evaluation available
     ContextPulse verified, KV revocation checked

  3. require Sky           → Draft accumulation and chat loop available
     No release path until Helios Agreement State = ALLOW

  4. require Méh₁n̥s        → Governance enforcement available
     SieveKernel::boot() registers DB + REST + GraphQL interceptors

  5. require Dheghom       → Structured storage available
     DAL mappers and repositories active
```

3. CLI and Non-HTTP Context

The platform is built around request context. WP-CLI and background cron jobs run outside the HTTP request lifecycle and have no device fingerprint, no ContextPulse, and no browser context. Without a defined fallback, ContextEngine::current() throws on CLI, breaking all maintenance scripts and cron jobs.

Rule 3.1 — CLI System Context: When PHP_SAPI === "cli", Sirus resolves a System-Level Context rather than throwing. The System Context carries fixed values:

```
identity_id  = "SYSTEM"
trust_score  = 1.0
trust_level  = "NORMAL"
authority_id = "GLOBAL"
device_id    = "CLI"
```

Rule 3.2 — CLI Agreement: Helios grants ALLOW_ORIGIN to the SYSTEM identity for all maintenance tasks, bypassing proof and Pulse verification. This bypass is available exclusively when identity_id === "SYSTEM" and PHP_SAPI === "cli". No other bypass path exists.

Rule 3.3 — CLI Governance: Méh₁n̥s governance (the Epistemic Sieve) does not evaluate Policy Packs for CLI operations. Direct DAL operations under SYSTEM identity are permitted for maintenance tasks. The CryptographicAuditLedger still records all writes under the SYSTEM identity.

4. Data Integrity — The Triple Binding Rule

Ownership enforcement in the existing specs is scattered: Sky enforces it at commit, Dheghom acknowledges it, and Helios defines session binding separately. The Triple Binding rule compresses these into one enforceable invariant that applies at every governed write.

Rule 4.1 — Triple Binding Requirement: Any write operation to aiwa_ or sparxstar_ prefixed meta keys must verify that the request aligns across three signals before any persistence logic executes:

- session_id — from the custom wp_spx_sessions table
- identity_id — from the Helios-verified request context (not from the ContextPulse, which does not carry identity claims)
- device_id — from the Sirus-issued ContextPulse

Rule 4.2 — Triple Binding Failure: If any one of the three signals is missing or mismatched, the Dheghom DAL must throw a TripleBindingException and halt the transaction. No partial write occurs. The session remains in its pre-write state. The failure is logged to the CryptographicAuditLedger.

Rule 4.3 — Triple Binding at Release: The triple binding check runs as the first step of the Release Gate (before decryption, before Policy Pack evaluation). A release attempt that fails triple binding never reaches the Sieve.

```
// Triple binding check — first step of every governed write
public function assertTripleBinding(
    string $sessionId,
    string $identityId,
    string $deviceId
): void {
    $session = $this->sessionRepository->findByUuid($sessionId);

    if ($session === null) {
        throw new TripleBindingException("Session not found: {$sessionId}");
    }
    if ($session->identity_id !== $identityId) {
        throw new TripleBindingException("Identity mismatch on session
{$sessionId}");
    }
    if ($session->device_id !== $deviceId) {
        throw new TripleBindingException("Device mismatch on session
{$sessionId}");
    }
}
```

5. Closed Open Items — Normative Defaults

The following items were marked [OPEN] in one or more product specifications. They are closed here as normative platform defaults. Product specifications must be updated to reflect these decisions.

5.1 Trust Score Algorithm

DECISION 5.1: The trust score algorithm is frozen at the following weights as the Standard Default for v1.0. These weights may be tuned in future versions but must not be changed without updating this document.

Signal	Weight	Trigger
Device continuity	40%	device_id stable, fingerprint consistent
Geographic consistency	20%	no ASN/country jump, geo zone stable
Session stability	20%	session is not new, no recent failures
Anomaly flags	20%	no behavior_flags set in current context

Computation: base score = 1.0. Each signal that fails deducts its weight. Score is clamped to [0.0, 1.0]. A drifting device deducts 0.3 (not 0.4) because drift is detected mid-session, not at first contact, and some drift is expected on mobile networks.

```
public function compute(
    DeviceRecord $device,
    EnvironmentRecord $env,
    SessionState $session
): float {
    $score = 1.0;
    if ($device->isDrifting())          { $score -= 0.3; }
    if ($env->isGeoMismatch())           { $score -= 0.2; }
    if ($session->isNew())               { $score -= 0.1; }
    if ($session->hasRecentFailures())   { $score -= 0.3; }
    return max(0.0, min(1.0, $score));
}
```

5.2 Signing Key Location

DECISION 5.2: Signing keys are stored as follows. This is a platform-wide requirement. Storing signing keys in WordPress options, the database, or any PHP-accessible persistent store is prohibited.

- Edge (Cloudflare Worker): Cloudflare Secrets Store. The key is never exposed to PHP.
- Origin (WordPress plugin / PHP): Environment variables only. Set at the server/VIP environment level, not in wp-config.php or any committed file.
- Key rotation policy: both edge and origin must be updated atomically. A mismatch between edge and origin signing keys will cause all pulse verification to fail, which is intentional — the system fails closed.

5.3 Cache Backend

DECISION 5.3: A persistent object cache (Redis or Memcached) is a platform system requirement for trust-state continuity. In environments without a persistent object cache, the trust cache layer is disabled and every request falls back to per-request pulse verification only. Transients must not be used for trust state — on sites without persistent cache, transient writes to the database at the frequency of trust checks would cause performance failure.

5.4 visitorId Source

DECISION 5.4: visitorId is a client-side persistent identifier derived from JS signal capture (browser fingerprint signals including canvas hash, screen resolution, timezone, and user agent). It is probabilistic metadata used exclusively for device drift detection. It is not a security primitive. The server-issued device_id is the real persistent identity. The fingerprint_hash formula is: sha256(visitorId + userAgent + ipSubnet). If visitorId changes between visits (browser update, clearing site data), the session recovery flow attempts a server-side fingerprint match before treating the device as new.

5.5 Pulse TTL Values

DECISION 5.5: The following TTL values are the working defaults pending real-world testing in low-connectivity environments. They must not be hardcoded — they must be configurable via environment variable or filter so they can be adjusted without a deployment.

Context	TTL	Notes
High-connectivity, stable	60–120 seconds	Standard window
Low-connectivity (2G, rural) — Level 1 only	Up to 10 minutes	Edge serves from cache, no origin round-trip
Medium sensitivity (Level 2)	30–60 seconds	Tighter window before step-up
High sensitivity / sovereign (Level 3)	15–30 seconds	Maximum freshness required

[OPEN] Exact durations require real-world testing in African low-connectivity environments. These values are the working defaults, not the final values.

5.6 UEC Migration Strategy

DECISION 5.6: UEC (sparxstar-user-environment-check) is in production. Migration follows the Transparent Proxy pattern:

1. Sirus boots as mu-plugin and populates its internal environment record on every request.

2. The UEC compatibility shim (final class StarUserEnv) proxies all existing method calls to Sirus internals. No calling code changes.
3. For the first 30 days post-deployment, the fail-closed governance rules apply only to new /sparxstar/ and /aiwa/ endpoints. Existing UEC-consuming code paths continue to function.
4. After the 30-day stabilisation window, fail-closed rules extend to all governed paths. UEC shim remains permanently as the frozen public interface.

The UEC class names and method signatures are frozen and must never change. This is a permanent compatibility guarantee, not a migration artifact.

6. Platform Ownership Map

This table is the single authoritative reference for which system owns each concern. When a developer is uncertain where a piece of logic belongs, this table governs. If the concern is not in this table, it belongs in the product spec of the layer closest to it in the execution order.

Concern	Owner	Not owned by
Device registration and device_id issuance	Sirus	Helios, Sky, Dheghom
Trust score computation	Sirus (TrustEngine)	Helios, Mḗhₒṇs
ContextPulse generation and signing	Sirus (PulseGenerator)	Helios
ContextPulse verification	Helios (PulseVerifier)	Sirus, Sky
Agreement evaluation (proceed / deny)	Helios (AgreementEvaluator)	Sirus, Mḗhₒṇs
KV revocation list reads and writes	Helios (KVRevocationClient)	Sirus
Step-up trigger logic	Helios (StepUpPolicy)	Sirus, Sky
Draft accumulation (conversation state)	Sky (wp_spx_sessions)	WordPress sessions
Commit Gate enforcement	Sky (ReleaseGateService)	Dheghom
Governance policy evaluation	Mḗhₒṇs (PolicyDispatcher)	Sky, Dheghom
GovernanceToken minting	Mḗhₒṇs (GovernanceTokenMinter)	Helios, Sky
GovernanceToken validation	Dheghom (TokenValidator)	Mḗhₒṇs
Structured field persistence	Dheghom (DAL)	Sky, Mḗhₒṇs
Cryptographic audit ledger	Mḗhₒṇs (CryptographicAuditLedger)	Dheghom
Identity claims (who is this person)	Sirus (IdentityResolver)	ContextPulse
Authority resolution	Sirus (AuthorityResolver)	Helios, Mḗhₒṇs
WordPress auth (wp-admin only)	WordPress core	Any Sparxstar component

7. Remaining Open Items

The following items are architecturally committed but not yet resolved. They are tracked here so their status is visible in one place. Each must be closed in this document before the affected product spec can be marked final.

[OPEN] 7.1 — Pulse TTL exact values: Specific durations per connectivity tier and resource sensitivity level require real-world testing in African low-connectivity environments. Working defaults are defined in Decision 5.5. Final values replace the working defaults when testing is complete.

[OPEN] 7.2 — Level 3 hardware key recovery quorum: Level 3 device recovery requires community co-signature. The number of keyholders required, the interface through which they act, and the governance process are not yet defined. This is a community/governance decision, not a technical spec decision, but the technical spec must support whatever process is defined.

[OPEN] 7.3 — Control plane security for Helios Admin UI: The Helios admin interface must itself be governed. How staff admin access is secured — what proof type is required, whether it runs through Helios itself, and what authority is required — is not yet specified.

[OPEN] 7.4 — TOTP offline fallback for Africa: Enrollment and recovery flow for users in genuinely low-connectivity environments where TOTP apps cannot sync. The UX and technical flow for this path are not yet defined.

[OPEN] 7.5 — KV propagation timing: Exact window between revocation written to Cloudflare KV and propagation to all edge nodes. Cloudflare typical propagation is under one second but this needs confirmation and documentation for the fail-closed guarantees to be precise.

[OPEN] 7.6 — Learned trust patterns: Whether and how Sirus learns normal behavior for a device/identity pair and flags deviations from that baseline. Architecturally desirable, not yet specified.

[OPEN] 7.7 — Multi-Briefcase federation: How Sirus context travels between distinct Briefcase deployments in direct federation. Not required for v1.0 but must be resolved before federated DVE deployments are enabled.

8. Required Updates to Product Specifications

The following changes must be made to product specifications to align them with this Integrity Map. Each change is a specific, locatable update — not a full rebuild.

Sirus — Update to v3.0

- Add: mu-plugin deployment clause — "Sirus must be deployed as a WordPress Must-Use plugin. It cannot be deactivated." (Rule 2.1)
- Add: CLI System Context — define ContextEngine::current() behavior when PHP_SAPI === "cli". (Rule 3.1)
- Close: trust score algorithm — absorb the TrustEngine computation from Decision 5.1. Remove [OPEN] tag.
- Close: signing key location — absorb Decision 5.2. Remove [OPEN] tag.
- Close: cache backend — absorb Decision 5.3. Remove [OPEN] tag.
- Close: visitorId source — absorb Decision 5.4. Remove [OPEN] tag.
- Close: pulse TTL values — absorb Decision 5.5 working defaults. Keep [OPEN] for exact values pending testing.
- Add: UEC migration contract — absorb Decision 5.6.
- Add: PulseVerifier contract — the six-check verification sequence (HMAC, expiry, skew, device_id presence, trust_level enum, trust_score bounds).
- Add: DeviceMatcher scoring thresholds — explicit match scoring (user agent 0.4, subnet 0.3, device type 0.2, timezone 0.1) and match thresholds (≥0.8 strong, ≥0.6 weak/step-up, <0.6 no match).
- Add: StepUpPolicy — mandatory for all Level 3, mandatory for Level 2 when trust_score < 0.7.

Helios — New Unified Spec v1.0

- Add: HeliosClientInterface as a hard contract — "All enforcement points must depend only on this interface."
- Add: Selective enforcement rule — "Helios enforcement is opt-in via possession of a Helios-issued credential. Normal WordPress traffic not carrying a Helios auth cookie is not intercepted."
- Add: AgreementEvaluator — the canonical evaluateAgreement() function identical in TypeScript (edge) and PHP (origin).
- Add: PulseVerifier implementation contract.
- Add: TrustEngine computation (from Decision 5.1).
- Add: DeviceMatcher scoring thresholds.
- Add: KVRevocationClient interface.
- Add: StepUpPolicy implementation.
- Add: Audit log entry schema (JSON structure for every decision log entry).
- Absorb: all open items resolved in this document.

Sky — Targeted Updates

- Remove: any reference to "Helios creates WordPress sessions." Replace with Rule 1.4 — continuity via ContextPulse device_id and Helios-verified identity context matched against wp_spx_sessions.
- Add: HeliosClientInterface dependency rule — "Sky must not perform identity or device checks through any path other than HeliosClientInterface."
- Add: selective enforcement note — "Sky sessions are gated only for requests carrying a Helios-issued credential. Normal WordPress traffic is unaffected."

Dheghom — Targeted Updates

- Add: Triple Binding as a required pre-condition for all governed writes (Rule 4.1, Rule 4.2).
- Add: TripleBindingException as a required exception class.
- Add: GovernanceToken as the absolute prerequisite for any released payload write — "No governed field write to a published record is permitted without a valid GovernanceToken verified by TokenValidator."
- Add: Helios selective enforcement note — "Helios enforcement applies only to requests carrying a Helios-issued credential. The Dheghom DAL must not assume all requests are Helios-governed."

---

9. Implementation Enforcement Rules

These rules are the last line of defence before build. Architecture is correct. Boundaries are defined. Contradictions are resolved. What breaks this system in production is not a design flaw — it is a developer collapsing layers for convenience. These rules make that collapse explicit, detectable, and prohibited.

If you shortcut the layers, you break the system. If you respect the layers, the system enforces itself.

Rule 9.1 — Schema Is Law (Sky to Dheghom boundary)

The SCF Ability Schema is the only permitted shape of reality. Sky extracts and produces candidate JSON. Dheghom decides what is allowed to exist. These are two different acts performed by two different layers. They must never be merged.

- Any field not in the registry must be dropped silently — not logged as an error, not stored in inferred_data as truth, not passed to any downstream system
- Any type mismatch must be rejected at the DAL ValidationEngine — not coerced, not approximated
- No dynamic schema extension is permitted at runtime — the registry is the only permitted source of field definitions

Without this: schema drift happens silently, data becomes non-queryable, and governance becomes unenforceable because the policy rules reference fields that no longer exist in a consistent form.

Rule 9.2 — Asset Immutability Is Absolute (Sky and Dheghom)

The asset field — the elder's words, the artist's recording, the contributor's story — is never overwritten, never normalized beyond safe storage, never merged with derived output. This is not a preference. It is the legal backbone of the system's promise to every contributor.

- Asset fields must never be overwritten after first write
- Asset fields must never be normalized, cleaned, or improved by any automated process
- Any modification to asset content must produce a new field with a _derived, _cleaned, or _translated suffix — the original is never touched
- The raw version is always preserved exactly as received, in the field it arrived in

Break this once and trust collapses permanently. What she gave is what is stored. That sentence is the entire legal and ethical foundation of the DVE.

Rule 9.3 — The ContextPulse Is Not Identity (Sirus to Helios to Sky)

The ContextPulse carries device state, trust signal, and context snapshot. It does not carry user_id, authority decisions, or permissions. Any code that resolves a user's identity from the Pulse alone is wrong and creates a replay attack surface.

- Never resolve user identity from the ContextPulse alone
- Never use the Pulse for authorization decisions — pair it with Helios verification
- Sky session lookup must use device_id from the Pulse to locate the session, then resolve identity separately through the Helios-verified request context

If violated: a stolen Pulse becomes a stolen identity. Device spoofing becomes identity spoofing. The entire trust model collapses to a single stolen cookie.

Rule 9.4 — Draft Is Never Safe to Interpret (Sky and Mḗh₁n̥s)

Draft data is incomplete, unverified, and potentially contradictory. It is a conversation accumulation store, not a governed record. The Sieve runs at Release, not at Save. Draft is protected, not validated.

- Mḗh₁n̥s must never run Policy Pack evaluation on draft records
- The AI pipeline must never read draft records — the training SELECT enforces this via post_status = publish AND governance_token IS NOT NULL
- Derived layers must never treat draft field values as truth — they are candidate data awaiting human confirmation

If developers shortcut this: governance leaks upstream into the conversation loop, the AI trains on unvalidated data, and the patent boundary between intake and archive collapses.

Rule 9.5 — Edge First, Origin Last (Helios)

Every request must follow the sequence: Edge decision first, Origin decision only if required. Calling the origin for every request destroys latency guarantees and makes the system unusable on the 2G connections it is specifically designed for.

- Edge-Helios must reject invalid pulses, serve Level 1 content from cache, and never call origin for Level 1
- Origin-Helios must only run for Level 2 and Level 3 requests, and for step-up authentication flows
- Any developer shortcut that calls origin for Level 1 verification is a performance violation and must be caught in code review

The users in The Gambia, Nairobi, and rural tribal communities are not edge cases. They are the primary audience. If the system requires a US origin round-trip to load a public profile, the DVE is broken for the people it exists to serve.

Rule 9.6 — No Silent Failure (All Layers)

The system must never fail invisibly. An invisible failure is worse than a hard error — it produces incorrect state with no signal that anything went wrong, making governance appear arbitrary and debugging impossible.

- Every Helios agreement evaluation must resolve to exactly one of four values: ALLOW_EDGE, ALLOW_ORIGIN, STEP_UP, or DENY. No null return, no fallback value,

no default pass. Every failure must return a deterministic state: DENY, STEP_UP, QUARANTINE, or ValidationException

- Every governance decision — pass or fail — must be logged to the CryptographicAuditLedger (Mḗh₁n̥s) or the system log (non-governance failures)
- Exception: schema-unknown fields are dropped silently by design (Rule 9.1). This is the only permitted silent operation in the system.
- Error messages must be deterministic and attributable — the same input must always produce the same failure message

Rule 9.7 — Governance Before Intelligence (Mḗh₁n̥s to AI pipeline)

The AI never sees ungoverned, unverified, draft, or quarantined data. This is the ethical boundary, the legal boundary, and the Patent Family C boundary. It must never be bypassed under any circumstances, including development, testing, or performance optimization.

The AI training pipeline query is fixed and inviolable:

```
SELECT * FROM posts WHERE post_status = 'publish' AND governance_token IS NOT NULL
AND token_expiry > NOW()
```

- No query against governed content may omit the post_status = publish condition
- No query against governed content may omit the governance_token IS NOT NULL condition
- No administrative shortcut may bypass this query for any purpose including seeding, testing, or backfilling

This is your ethical boundary, your legal boundary, and your patent boundary simultaneously. Bypass it once in a test environment and it will be bypassed in production. Any write that results in post_status = publish must originate from the Release Gate only. A direct wp_update_post() call that sets post_status to publish on a governed record, from any path other than ReleaseGateService, is a governance breach and must be caught by the DatabaseWriteInterceptor.

Rule 9.8 — NFC Normalization Is a Design Constraint, Not a Feature (Dheghom)

This system serves oral cultures and languages being digitized for the first time. Two contributors may type the same word using different keyboard layouts — a combined character versus a base character with a separate accent mark. Without NFC normalization, the system stores two different strings and cannot recognize them as the same word. Search fails. Archival bridging fails. Linguistic analysis produces false negatives.

- Unicode NFC normalization (Normalization Form C) must be applied at the DAL validation layer via ValidationHelper::normalize_unicode_nfc() before any string field is stored
- NFC normalization applies to all linguistic fields — it does not apply to hashes, UUIDs, or binary fields

- This is not optional for languages that use combining characters — failure produces silent data erasure where two records of the same word are stored as different words and never reconciled

Rule 9.9 — QUARANTINE Is a Success State (Méh₁n̥s)

In a standard CMS, a save error is a failure. In the DVE, QUARANTINE is a successful outcome. It means the system correctly identified data that cannot yet be released and preserved it safely for Elder review. A developer who sees a QUARANTINE result and attempts to fix it so the save goes through has broken the system.

- QUARANTINE is the Patent Family C requirement that protects sacred and sensitive data from AI ingestion until an Elder reviews it
- A QUARANTINE result must never be treated as a bug to be resolved by adjusting code — it is a policy outcome to be resolved by a human reviewer
- The audit trail of a QUARANTINEd record is as important as the record itself — it must be preserved intact
- If a developer adds logic to retry or reroute a QUARANTINEd write, they are bypassing sovereign governance law

Rule 9.10 — No Direct Database Writes to Governed Fields (Dheghom and Helios)

Because Sirus and Helios sit at the edge and kernel layers, developers may be tempted to use $wpdb->update() or direct SQL to bypass the overhead of the DAL and Helios verification. This is an absolute prohibition. It is not a performance optimisation — it is a security breach and a violation of the Indigenous Data Sovereignty contract.

- Any write to an aiwa_ or sparxstar_ prefixed field that does not pass through the Dheghom Repository is a security breach
- Any write to a governed field that does not carry a valid GovernanceToken verified by TokenValidator is a security breach
- WP-CLI scripts, migration tools, and seeding scripts are not exempt — they must route through the DAL under the SYSTEM identity
- The DatabaseWriteInterceptor exists specifically to catch this failure mode. If it is firing on legitimate code, the code is wrong, not the interceptor. A GovernanceToken is valid for one release transaction only. It cannot be reused across payloads, sessions, or requests. Token reuse is treated identically to token forgery — the write is rejected and the attempt is logged.

Rule 9.11 — Context Must Exist Before Execution (Sirus hard guarantee)

ContextEngine::current() must return a fully populated, valid SirusContext object or throw immediately. It must never return a partially constructed context, a null, or a default-empty object that allows execution to proceed silently with undefined device_id, trust_score, or authority_id.

- ContextEngine::current() returns a valid SirusContext or throws a ContextBootException — no other outcome is permitted
- A partial context — one where device_id is present but trust_score is missing, or authority_id is null in a non-anonymous tier — is not a valid context. It must throw.
- Code that calls ContextEngine::current() and continues execution without checking for a thrown exception is incorrect and will fail PHPStan Level 5. A ContextBootException must terminate the request. It must never be caught and swallowed. A try/catch block around ContextEngine::current() that does not re-throw is a prohibited pattern — if the system continues without context, every downstream decision (Helios agreement, Triple Binding, governance evaluation) operates on undefined state and fails silently. There is no recovery from a missing context. There is only halt.
- The CLI System Context (Rule 3.1) is the only context that may carry fixed values — it is still a complete, valid object

If context is partial: Helios agreement decisions become undefined, Triple Binding checks become meaningless, and governance enforcement operates on fabricated signals. A partial context is more dangerous than no context.

Rule 9.12 — Draft Encryption Is Mandatory (Dheghom)

All draft payloads must be encrypted using AES-256 envelope encryption before any database write. This is not a security preference — it is the cryptographic guarantee that makes the system's promise to contributors enforceable. A plaintext draft in the database means Sparxstar engineers can read it. That breaks the guarantee permanently.

- Any draft save operation that attempts to write plaintext payload to the database must throw a DraftEncryptionException and block the save
- The $isDraft flag and the EncryptionEngine middleware are not optional — they are part of the save() contract for all governed field writes in draft state
- Temporary plaintext storage "for debugging" or "for performance" is prohibited. There is no grace period for this rule.
- Released payloads (post_status = publish, with valid GovernanceToken) are stored in clear after the Release Gate has verified them — this is by design and does not violate this rule

The moment a plaintext draft exists in the database, the system's statement "Sparxstar engineers cannot read it" is a lie. That lie, once told, cannot be untold.

Rule 9.13 — Namespacing Is Detection, Not Protection (All Layers)

Field prefixes (aiwa_, sparxstar_) identify governed fields. They do not protect them. Any WordPress plugin — WooCommerce, Post Grid, AI Engine, or any arbitrary helper — can call get_post_meta() or construct a WP_Query against governed fields, and WordPress will return the data without restriction. Detection of a violation after the fact is not the same as preventing it.

Isolation in WordPress is not given — it is enforced. The write interceptors (DatabaseWriteInterceptor) block unauthorized writes. The read interceptors

(MetadataReadInterceptor and QueryEnforcementFilter) block unauthorized reads. Both are required. Neither alone is sufficient.

- MetadataReadInterceptor must hook the get_post_metadata filter and return null for any attempt to read aiwa_ or sparxstar_ fields on a draft or quarantined record from outside the Dheghom DAL
- QueryEnforcementFilter must hook posts_where and enforce the AI pipeline query constraint (post_status = publish AND governance_token IS NOT NULL) on all WP_Query calls against governed post types from outside the DAL
- Both interceptors must be registered in SieveKernel::boot() at the same priority as the write interceptors — they are not optional and cannot be conditionally loaded
- A rogue plugin calling get_post_meta() on an encrypted draft record must receive null — not the ciphertext, not an error that reveals the field exists

Returning null on an unauthorized read is intentional. It is not a bug. It is the enforcement of the read boundary. A plugin that receives null for a governed field it should not access will fail gracefully. A plugin that receives the encrypted ciphertext has learned the field exists and its encrypted value — both are information leaks.

10. Platform Coding Standards

These standards apply to all six repositories — Sirus, Helios, Sky, Méh₁n̥s, Dheghom, and Ouroboros Integrity — and to all tooling, CI pipelines, and integrations written against this platform. They are not preferences. They are the minimum required for a system that must hold its guarantees in a hostile commercial plugin environment (Section 0).

"Loader enforces order, modules register behavior, assets are built, WordPress only renders."

10.1 PHP Coding Standards

Rule 10.1.1 — No Global Functions: No global PHP functions unless WordPress explicitly requires a callable (add_action, add_filter). When WordPress requires a callable, use a static class method, not a function declaration.

Correct:

```
add_action('init', [SPX\\Sky\\Bootstrap::class, 'init']);
```

Prohibited:

```
function spx_sky_init() { ... }
```

Rule 10.1.2 — final by Default: All classes are declared final unless they are explicitly designed for extension. Abstract base classes (AbstractRepository, AbstractModuleMapper) are the only permitted non-final classes. No inheritance except for defined base patterns. Prefer composition and interfaces over inheritance.

Rule 10.1.3 — Namespace Everything: All classes, interfaces, enums, and traits must live in a namespace under SPX\\{Layer}. No unnamespaced class is permitted. No class may be defined in the global namespace.

```
namespace SPX\\Sirus;  // correct

class ContextEngine { ... }  // prohibited — no namespace
```

Rule 10.1.4 — Symbol Isolation: All classes, functions (where unavoidable), hooks, constants, globals, options keys, transient keys, and database table names must be prefixed with SPX_ or namespaced under SPX\\. No unscoped symbol is permitted. Collision with third-party code is a security risk in a hostile plugin environment, not merely a style concern.

Rule 10.1.5 — No Logic Outside Classes: Procedural code at file scope is prohibited except for: the require_once calls in 00-sparxstar-loader.php, the defined('ABSPATH') || exit guard, and Bootstrap::register() calls in the loader. All other logic lives in class methods.

Rule 10.1.6 — Strict Types: Every PHP file must begin with declare(strict_types=1). No exceptions.

Rule 10.1.7 — PHPStan Level 5: PHPStan Level 5 must pass before any merge to main. This is a CI gate, not a recommendation. Null-safe operator (?->) required wherever a nullable is consumed.

10.2 Module Bootstrap Pattern

Each module exposes a Bootstrap class as its single public entry point. The loader calls Bootstrap::register() for each module in sequence. Modules never require each other directly — only the loader wires them.

```
namespace SPX\\Sky;

final class Bootstrap {

    public static function register(): void {

        // register REST routes, hooks, services

    }

}
```

10.3 Hook and Filter Naming

Rule 10.3.1 — Hook Prefix: All WordPress hooks registered by this platform must follow the pattern: spx_{layer}_{action}. No hook may be registered without the spx_ prefix.

```
spx_helios_agreement_result

spx_dheghom_before_write

spx_sirus_context_resolved
```

10.4 Template Rules

Rule 10.4.1 — Templates Are Dumb: Templates are a render layer only. They echo variables. They contain basic conditionals on pre-resolved data. They contain nothing else.

Prohibited in templates:

- **PROHIBITED: Database queries of any kind**
- **PROHIBITED: Business logic or data transformation**
- **PROHIBITED: Helios agreement checks**
- **PROHIBITED: Governance evaluation**
- **PROHIBITED: Direct calls to any SPX service class**

The correct pattern: a controller or service resolves all data, passes a typed view model to the template, and the template renders it.

Rule 10.4.2 — Template Location: Templates live in /wp-content/themes/spx-theme/templates/. No template logic lives in mu-plugins. No core logic lives in themes.

10.5 JavaScript Standards

Rule 10.5.1 — JS Has No Authority: JavaScript makes no security decisions, no identity decisions, no governance decisions, and no validation decisions that the server does not also make independently. JS collects input and sends requests. The server decides everything.

JS sends:

```
{ "text": "..." }
```

Server decides: schema, validation, identity, storage, governance.

Rule 10.5.2 — JS Namespace: All JavaScript must be namespaced under window.SPX. No unscoped JS globals.

```
window.SPX = window.SPX || {};
```

```
SPX.Sky = {};
```

Rule 10.5.3 — No Authority in JS: JS must never: determine trust state, determine identity, validate governance, generate authoritative identifiers (UUIDs, session IDs), or make decisions that the server does not independently verify.

Rule 10.5.4 — All Requests Carry the ContextPulse: Every request from JS to the server must include the Sirus-issued ContextPulse (HttpOnly cookie, sent automatically by the browser). JS must never attempt to construct, simulate, or bypass the pulse. Hidden inputs and JS-calculated values are never trusted by the server.

Rule 10.5.5 — ESNext + Built Assets Only: All JS is written in ESNext (ES2020+), compiled by the build toolchain. Raw JS is never deployed to production. No inline script tags. No global script pollution. All scripts enqueued via wp_enqueue_script() with version from the asset manifest.

10.6 CSS Standards

Rule 10.6.1 — Prefix All Selectors: All CSS classes, IDs, and custom properties must be prefixed with .spx-. No generic selectors, no global resets, no unscoped element selectors.

- **ALLOWED: .spx-sky-container { }**
- **ALLOWED: .spx-chat-box { }**
- **PROHIBITED: div { }**
- **PROHIBITED: .container { }**
- **PROHIBITED: * { margin: 0; }**

10.7 Asset Loading

Rule 10.7.1 — Enqueue Only: All JS and CSS must be loaded via wp_enqueue_script() and wp_enqueue_style(). No inline scripts, no direct script tags, no global enqueues from themes that load on every page regardless of context.

Rule 10.7.2 — Asset Manifest: JS builds must produce an asset manifest (index.asset.php via @wordpress/scripts or equivalent). Version and dependency hashes come from the manifest, not hardcoded strings.

11. Runtime Stack and Toolchain

These are the required and approved components of the platform stack. This list is exhaustive for the core stack. Any addition requires an explicit decision recorded in this document. The test for any addition: if a tool writes or decides, it must go through your layers — or it does not belong.

11.1 Core Runtime (Non-Negotiable)

- WordPress 6.9+ — host runtime for UI, routing, REST API, and admin
- PHP 8.2+ — required for readonly classes, fibers, intersection types, and strict null handling throughout the stack
- MariaDB 10.6+ (preferred) or MySQL 8.0+ — JSON column support, index performance, and stability required. SQLite and older MySQL versions are not supported.
- OPcache — required on all PHP environments. PHP execution without OPcache is not a supported configuration.

11.2 Edge Layer

- Cloudflare Workers (TypeScript) — Helios edge agreement evaluation, Level 1 content serving, pulse verification, KV reads
- Cloudflare KV — revocation list, device flags, session revocations. Security infrastructure, not cache. Read failure = DENY.

Hard rule: Edge can deny without origin. Origin cannot override an edge DENY. See Section 2 of this document and Ouroboros Integrity Section 2.

11.3 Cache Layer

- Redis via Predis (Human Made) — primary trust cache, context cache, short-lived token cache. Required for trust-state continuity.
- Memcached — optional secondary read cache only

Hard rule: Cache is acceleration only, never source of truth. Governed data is never stored in cache. See Platform Integrity Map Decision 5.3.

11.4 Schema Layer

- Secure Custom Fields (SCF) — schema registry and field definitions only. SCF defines the shape of governed fields. It does not write governed data directly.

SCF is a schema tool. It is not a write authority. All writes to governed fields route through the Dheghom DAL regardless of what SCF provides in its UI.

11.5 AI Layer

- AI Engine plugin — sandboxed to chat and extraction functions only. It listens and extracts. It does not save, validate, automate, or publish.
- **PROHIBITED: AI Engine writing to any governed field**
- **PROHIBITED: AI Engine triggering any publish or commit action**
- **PROHIBITED: AI Engine accessing Dheghom repositories directly**

The AI is a translation layer. Sky constrains its output to the Ability schema. Dheghom validates what Sky passes. AI Engine has no path to storage that does not flow through both.

11.6 Commerce Layer

- WooCommerce — isolated to product sales and payment processing. WooCommerce must not interact with governed fields, governed post types, or Dheghom repositories.
- **PROHIBITED: WooCommerce hooks writing to aiwa_ or sparxstar_ fields**
- **PROHIBITED: WooCommerce order processing triggering Dheghom writes**

WooCommerce operates entirely in the WordPress host runtime layer. It is isolated from the sovereign runtime by the DatabaseWriteInterceptor and MetadataReadInterceptor in Mḗhₒ̥ns.

11.7 Media Layer

- Imagick (server-side) — generates derived images (thumbnails, watermarked versions) only. Original media assets are immutable once ingested (Rule 9.2).
- **PROHIBITED: Imagick or any media tool overwriting the original asset field**

Derived images live in derived fields (_derived suffix). The original is never touched.

11.8 Display Layer (Controlled Use)

- Post Grid — display only. Must respect governance filters. Must not query governed fields outside the DAL. Must not expose draft or quarantined records.
- Cloudflare WordPress Plugin — cache purge integration only. Not a security component. Must not be relied upon for any governance or trust function.

11.9 What Is Explicitly Not Approved

The following categories of tool are not approved for this platform. Adding any of them requires a decision recorded in this document explaining why the addition does not violate Section 0 (Commercial Plugin Hostility).

- **PROHIBITED: ORM plugins or database abstraction layers that bypass the Dheghom DAL**
- **PROHIBITED: Form builder plugins that write directly to post meta**

- **PROHIBITED: AI automation plugins that trigger publishing, scheduling, or meta writes without explicit human action**
- **PROHIBITED: Meta helper plugins (ACF alternatives, custom field plugins) that write to governed fields**
- **PROHIBITED: Import/export plugins that operate on governed post types without routing through the DAL**
- **PROHIBITED: Any plugin that registers hooks at priorities that could intercept or interfere with SieveKernel interceptors**

11.10 Build Toolchain

PHP

- Composer 2.x — dependency management and PSR-4 autoloading. Each module has its own composer.json. Ouroboros Integrity is a required dependency of all modules.
- PHPStan Level 5 — static analysis. CI gate. Must pass before merge.
- PHP-CS-Fixer or PHPCS — PSR-12 enforcement. CI gate.

JavaScript and CSS

- @wordpress/scripts (Webpack + Babel + PostCSS) — standard build pipeline. Produces index.js and index.asset.php per module.
- ESLint with @wordpress/eslint-plugin/recommended — JS lint. CI gate.
- Stylelint with selector-class-pattern: ^spx-[a-z0-9\-]+$ — enforces .spx- prefix on all CSS selectors. CI gate.
- Prettier — formatting. Non-negotiable. Single quotes, semicolons, 100 character print width.

Testing

- PHPUnit — PHP unit and integration tests. Required for: AgreementEvaluator (shared test vectors), TokenValidator (three-gate verification), Triple Binding (all three signals), Release Gate (PASS, QUARANTINE, and DENY paths).
- Jest — JavaScript unit tests. Minimum: smoke tests on SPX.Sky extraction output shape.
- shared-test-vectors.json — owned by Ouroboros Integrity. Both TypeScript and PHP AgreementEvaluator test suites must import and pass this file. Identical inputs must produce identical outputs in both implementations.

CI/CD Pipeline (Required Gates — in order)

- composer install
- npm ci
- npm run build (assets must build cleanly)
- vendor/bin/phpstan analyse (Level 5, no errors)

- vendor/bin/phpcs (PSR-12, no errors)
- npm run lint (ESLint + Stylelint, no errors)
- vendor/bin/phpunit (all tests pass)
- Fail on any error. No warnings-only mode. No skip flags in production CI.

Versioning

- Semantic versioning: MAJOR.MINOR.PATCH across all repositories
- Build artifacts committed — no runtime asset compilation
- No direct commits to main — all changes via PR with CI green

11.11 Database Table Naming

Rule 11.11.1 — Sovereign Data Namespace: All SPX-controlled tables must use the spx_ prefix. WordPress table prefix (wp_) must not be used for SPX system tables. This avoids collision, avoids WordPress multisite prefix assumptions, and preserves the sovereignty boundary.

- spx_sessions — Sky conversation accumulation (not wp_spx_sessions)
- spx_records — Dheghom governed records
- spx_tokens — GovernanceToken store
- spx_audit — CryptographicAuditLedger
- spx_sirus_devices — Sirus device records
- spx_helios_revocations — KV mirror for origin-side revocation checks

WordPress is not authority over SPX data schema. SPX tables are created and managed by Ouroboros Integrity migrations, not WordPress dbDelta calls on wp_ prefixed tables.

11.12 ai_manifest

Each repository maintains an ai_manifest.json — a machine-readable registry of every symbol (class, interface, enum, constant, hook) the repository exposes. This serves as build-time collision detection and developer reference.

```
{
  "symbol": "SPX_SIRUS_ContextEngine",
  "type": "class",
  "owner": "sirus",
  "path": "/mu-plugins/sparxstar-sirus/src/ContextEngine.php"
}
```

- Build-time validation: no duplicate symbols across all six manifests
- CI gate: manifest must be updated when new symbols are added — stale manifest fails CI

- Runtime debug check (development environments only): optional startup verification that loaded symbols match the manifest

---

One rule that governs all tooling decisions: if a tool writes or decides, it must go through your layers — or it does not belong. The stack listed above is the full approved stack. Everything else is a decision that requires justification against Section 0.

The Digital Village Elder is not a WordPress plugin. It is a Sovereign Knowledge Vault that happens to use WordPress as a runtime. Sirus is the senses. Helios is the will. Sky is the voice. Mḗhₒns is the law. Dheghom is the memory. If any one layer is collapsed into another for developer convenience, the cryptographic guarantee — and the trust of the communities this system was built to serve — is lost.

This document is the platform's source of truth for cross-component decisions. It is updated whenever an [OPEN] item is resolved or a new contradiction is identified. All other documents defer to it.